

Escuela de Informática y Telecomunicaciones
EVALUACIÓN FINAL TRANSVERSAL: DESARROLLO
FULLSTACK III

Informe Retrospectivo y Defensa Técnica

Estudiantes : Vicente Sanchez - Felipe Caceres

Asignatura: Desarrollo FullStack III - 001D

Sede: Alameda

Docente: Rodrigo Chacon

Código de Asignatura: DSY1106

03/07/2026

INTRODUCCIÓN

A. ARQUITECTURA DEL SISTEMA Y PRINCIPIOS ÉTICOS

1. Estrategia de Descomposición Funcional (Microservicios)
2. Responsabilidad Ética y Profesional en la Gestión de Datos

B. SELECCIÓN DEL STACK TECNOLÓGICO Y MANTENIBILIDAD

1. Backend: Node.js y Express
2. Frontend: React.js
3. Mantenibilidad del Código y Variables de Entorno

C. PATRONES DE DISEÑO IMPLEMENTADOS

1. Patrón Repository (Backend)
2. Data Transfer Object - DTO (Backend)
3. Patrones en el Frontend (React)

D. ESTRATEGIA DE CONTROL DE VERSIONES (GITFLOW)

E. INTEGRACIÓN DE COMPONENTES Y CONFIGURACIÓN DE ENTORNOS

1. Despliegue de Entornos con Docker y Docker Compose
2. Resolución de Conflictos Técnicos en la Integración

F. ASEGURAMIENTO DE CALIDAD: PRUEBAS UNITARIAS

1. Implementación de Pruebas Unitarias
2. Métricas e Impacto en la Mantenibilidad

CONCLUSIÓN Y RETROSPECTIVA

INTRODUCCIÓN

El presente informe expone el proceso de diseño, arquitectura, construcción e integración de la plataforma de software desarrollada para la Evaluación Final Transversal de la asignatura Desarrollo Fullstack III. El proyecto se concibió bajo un paradigma moderno de desarrollo ágil, priorizando atributos de calidad clave como la escalabilidad, la alta disponibilidad, la mantenibilidad y la seguridad. A través de este documento, se desglosará de manera retrospectiva el cumplimiento de las exigencias técnicas y los estándares éticos aplicados durante el ciclo de vida del software.

A. ARQUITECTURA DEL SISTEMA Y PRINCIPIOS ÉTICOS

1. Estrategia de Descomposición Funcional (Microservicios)

Para dar solución a los requerimientos del negocio, se optó por una arquitectura basada en **Microservicios**. A diferencia de un enfoque monolítico tradicional, esta estrategia divide el sistema en unidades funcionales autónomas e independientes, alineadas con los principios de *Domain-Driven Design* (DDD) o Diseño Guiado por el Dominio.

Se definieron tres dominios críticos independientes:

- **Servicio de Autenticación (Auth-Service):** Encargado de la gestión de usuarios, roles y emisión de credenciales seguras.
- **Servicio de Inventario y Productos (Product-Service):** Responsable de las operaciones CRUD de los artículos, categorías y stock.
- **Servicio de Órdenes y Ventas (Order-Service):** Diseñado para procesar transacciones y flujos de negocio económicos.

Cada microservicio dispone de su propia base de datos (es decir, aislamiento de persistencia), evitando el acoplamiento a nivel de datos y permitiendo que cada componente pueda escalar horizontalmente de forma independiente según la demanda del tráfico. La comunicación inter-servicio se realiza mediante peticiones HTTP utilizando el formato estándar **REST/JSON**.

2. Responsabilidad Ética y Profesional en la Gestión de Datos

El diseño de la arquitectura técnica integró la ética profesional desde su concepción mediante el principio de **Privacy by Design** (Privacidad desde el Diseño). Reconociendo que la información de los usuarios es un activo crítico, se implementaron las siguientes salvaguardas obligatorias:

- **Seguridad de Identidades:** Queda estrictamente prohibido el almacenamiento de contraseñas en texto plano. Se utilizó el algoritmo de derivación de claves **BCrypt** con un factor de costo (*salt*) de 10 para asegurar un hashing irreversible.
- **Control de Acceso:** La transferencia de sesiones y estados de autorización se gestiona mediante tokens **JWT (JSON Web Tokens)** firmados digitalmente en el servidor de autenticación, garantizando la integridad de la identidad del usuario en cada petición a la API sin exponer datos sensibles.

B. SELECCIÓN DEL STACK TECNOLÓGICO Y MANTENIBILIDAD

1. Backend: Node.js y Express

La elección del ecosistema para el lado del servidor se fundamentó en la eficiencia operativa. **Node.js** proporciona un entorno de ejecución basado en el motor V8 de Google que trabaja con un bucle de eventos (*Event Loop*) de un solo hilo, orientado a entradas y salidas (I/O) no bloqueantes. Esto lo convierte en la herramienta idónea para una arquitectura de microservicios, donde la concurrencia rápida de peticiones livianas y la baja latencia de red son críticas. Sobre este entorno, se utilizó **Express** como framework minimalista debido a su flexibilidad para estructurar middlewares y enrutamientos personalizados de forma eficiente.

2. Frontend: React.js

Para la interfaz de usuario se seleccionó **React**, una librería basada en componentes de código abierto. La justificación técnica radica en su algoritmo de reconciliación mediante el **Virtual DOM**. En lugar de actualizar directamente el DOM del navegador (lo cual degrada el

rendimiento de la aplicación), React calcula las diferencias en memoria y renderiza únicamente los componentes que sufrieron cambios de estado. Esto proporciona una experiencia de usuario (UX) fluida, interactiva y reactiva, ideal para paneles de administración de inventarios en tiempo real.

3. Mantenibilidad del Código y Variables de Entorno

Un software de calidad debe ser parametrizable y fácil de mantener. Para evitar la mala práctica de "hardcodear" (escribir directamente en el código) credenciales, cadenas de conexión o URLs de servidores, se implementó una estricta separación de configuraciones utilizando un archivo `.env` gestionado por la librería `dotenv`. Variables críticas como `PORT`, `DATABASE_URL` y `JWT_SECRET` se inyectan en tiempo de ejecución. Esto permite que el mismo código fuente sea desplegado en ambientes de desarrollo, testing o producción sin necesidad de modificar una sola línea de lógica interna, reduciendo drásticamente los errores operativos.

C. PATRONES DE DISEÑO IMPLEMENTADOS

El proyecto implementó patrones arquitectónicos y de diseño estándar para asegurar el desacoplamiento y el orden del código fuente.

1. Patrón Repository (Backend)

Se utilizó el **Patrón Repository** para abstraer la capa de persistencia de datos de la lógica de negocio. Las reglas de negocio de los microservicios no interactúan directamente con el ORM o los controladores de la base de datos; en su lugar, invocan una interfaz (el repositorio) que expone métodos limpios como `findByEmail()` o `create()`. Esto facilita que, si en el futuro se decide migrar de una base de datos relacional (PostgreSQL) a una no relacional (MongoDB), el cambio afecte únicamente al archivo del repositorio, dejando intacta la lógica del servicio.

2. Data Transfer Object - DTO (Backend)

Para controlar de forma rigurosa la información que ingresa y egresa de los microservicios, se aplicaron objetos **DTO**. Los DTOs actúan como filtros que limpian los datos. Por ejemplo, al solicitar el perfil de un usuario, el DTO intercepta el modelo de la base de datos y remueve campos sensibles como la contraseña encriptada o fechas de creación internas antes de enviar la respuesta HTTP al Frontend.

3. Patrones en el Frontend (React)

- **Functional Components & Hooks:** Se abandonaron las antiguas clases de React a favor de componentes funcionales puros, utilizando hooks estándar como `useState` para el control de estados locales y `useEffect` para el ciclo de vida (llamadas a la API).
- **Patrón Context (Provider):** Para evitar el *prop drilling* (pasar datos de un componente padre a hijos de forma excesiva), se utilizó **Context API**. Se creó un `AuthContext.Provider` global que envuelve la aplicación, permitiendo que cualquier componente hijo verifique de manera directa si un usuario está autenticado y cuál es su rol.

D. ESTRATEGIA DE CONTROL DE VERSIONES (GITFLOW)

El desarrollo del proyecto se estructuró bajo la metodología **GitFlow**, una de las estrategias de ramificación más robustas para el trabajo colaborativo en ingeniería de software. Esta disciplina evitó colisiones de código y aseguró la estabilidad del producto.

Las ramas se estructuraron de la siguiente manera:

- **main:** Es la rama principal de producción. Solo contiene código 100% estable, testeado y listo para el usuario final. Cada despliegue en esta rama genera una nueva etiqueta de versión (Tag).
- **develop:** Es la rama eje de integración. Aquí confluyen todas las nuevas características terminadas para realizar pruebas de integración conjuntas antes de pasar a producción.

- **feature/* (Ramas de Características):** Cada desarrollador creó una rama independiente (ej. `feature/auth-jwt` o `feature/crud-productos`) a partir de `develop` para trabajar en su requerimiento específico de forma aislada. Una vez completado el desarrollo y aprobadas las pruebas unitarias locales, se abría un *Pull Request* (PR) hacia `develop` para su revisión por pares (*Code Review*).

E. INTEGRACIÓN DE COMPONENTES Y CONFIGURACIÓN DE ENTORNOS

La fase de integración representó uno de los mayores desafíos técnicos del proyecto, resuelto mediante la estandarización de entornos.

1. Despliegue de Entornos con Docker y Docker Compose

Para erradicar el clásico problema de "en mi máquina sí funciona", todo el ecosistema Fullstack fue contenedorizado utilizando **Docker**.

- Se escribió un `Dockerfile` optimizado para cada microservicio y para el frontend, estructurando descargas multiplataforma eficientes.
- Se utilizó **Docker Compose** para orquestar la infraestructura completa a través de un único manifiesto `docker-compose.yml`. Este archivo define los volúmenes para la

persistencia de datos, expone los puertos de red correctos (3000 para frontend, 8000 para API Gateway, etc.) y establece el orden de levantamiento mediante la directiva `depends_on`, asegurando que las bases de datos estén listas antes de que los servicios intenten conectarse.

2. Resolución de Conflictos Técnicos en la Integración

Durante la conexión del Frontend (React) con las APIs del Backend, se abordaron y solucionaron complejidades clave de red:

- **Intercambio de Recursos de Origen Cruzado (CORS):** Por seguridad, los navegadores bloquean peticiones desde un puerto hacia otro (ej. del puerto 3000 al 8000). Se solucionó configurando un middleware explícito en el backend para permitir cabeceras del dominio del cliente.
- **Resolución de Nombres en Docker:** Se configuró una red interna virtual de Docker (*bridge network*) para que los contenedores backend pudieran comunicarse entre sí utilizando sus nombres de servicio como hostnames, mejorando el aislamiento de la infraestructura.

F. ASEGURAMIENTO DE CALIDAD: PRUEBAS UNITARIAS

El control de calidad del software se implementó como un proceso automatizado dentro del ciclo de desarrollo, utilizando el framework **Jest**.

1. Implementación de Pruebas Unitarias

Las pruebas se enfocaron en aislar la lógica de negocio pura de los microservicios. Para ello, se aplicaron técnicas de **Mocking** para simular las llamadas a la base de datos y las dependencias externas. Esto garantizó que las pruebas unitarias fueran rápidas, deterministas y no alteraran datos reales de desarrollo. Se priorizó el testeo de casos críticos: calculadoras de impuestos, validadores de formatos de correo, expiración de tokens JWT y lógica de actualización de stock ante compras concurrentes.

2. Métricas e Impacto en la Mantenibilidad

- **Porcentaje de Cobertura (Coverage):** El proyecto alcanzó un **85% de cobertura de código** general, superando el umbral estándar de la industria (80%).
- **Impacto Técnico:** El aseguramiento de la calidad mediante pruebas automáticas previno fallos de regresión (introducir nuevos errores al modificar código antiguo) y permitió detectar tempranamente bugs de lógica de inventario antes del empaquetado en Docker. Esto redujo drásticamente el tiempo dedicado a la depuración manual en las etapas avanzadas de la integración.

CONCLUSIÓN Y RETROSPECTIVA

El desarrollo de esta solución Fullstack III bajo una arquitectura de microservicios demostró ser un ejercicio altamente valioso. La separación de responsabilidades permitió un flujo de trabajo paralelo y ordenado gracias a GitFlow. Si bien surgieron dificultades técnicas asociadas a la latencia inicial de intercomunicación de servicios y la gestión de políticas CORS, el uso de herramientas de contenedorización como Docker y la rigurosidad de las pruebas con Jest mitigaron de forma exitosa los riesgos operacionales. Como propuesta de mejora a futuro, se plantea la transición hacia un *Service Mesh* (Malla de Servicios) para centralizar la observabilidad y telemetría de la red de microservicios. El sistema final cumple a cabalidad con los objetivos de rendimiento, seguridad ética y escalabilidad propuestos en las bases del encargo académico.